

Lempel –Ziv Coding

- No requirement of prior probabilities
- Code length of symbols representing one message unit is fixed.
- Several variant of LZ algorithm
- LZ coding is done by writing location or index in binary followed by innovation digit
- Zip and gzip use variations of the Lempel-Ziv algorithm.

The Lempel-Ziv Algorithm

- The compression of an arbitrary sequence of bits is possible by coding a series of 0's and 1's as some *previous* such string (the prefix string) plus one new bit.
- Then, the new string formed by adding the new bit to the previously used prefix string becomes a potential prefix string for future strings.
- These variable length blocks are called **phrases**.
- The phrases are listed in a dictionary which stores the existing phrases and their locations.
- In encoding a new phrase, we specify the location of the existing phrase in the dictionary and append the new letter.

- Wish to code string 101011011010101011

We will begin by **parsing** it into comma-separated phrases that represent strings that can be represented by a previous string as a prefix, plus a bit.

The first bit, a 1, has no predecessors, so, it has a null prefix string and the one extra bit is itself:

1, 01011011010101011

The same goes for the 0 that follows since it can't be expressed in terms of the only existing prefix:

1, 0, 1011011010101011.

So far our dictionary contains the strings '1' and '0'.

Next we encounter a 1, but it already exists in our dictionary. Hence we proceed further. The following 10 is obviously a combination of the prefix 1 and a 0, so we now have:

1, 0, 10, 11011010101011

Continuing in this way we eventually parse the whole string as follows:

1, 0, 10, 11, 01, 101, 010, 1011

Now, since we found 8 phrases, we will use a three bit code to label the null phrase and the first seven phrases for a total of 8 numbered phrases (with the ninth and last phrase we found being expressed in the others, hence not needing to be numbered).

1, 0, 10, 11, 01, 101, 010, 1011

Now, since we found 8 phrases, we will use a three bit code to label the null phrase and the first seven phrases for a total of 8 numbered phrases (with the ninth and last phrase we found being expressed in the others, hence not needing to be numbered).

Next, we write the string in terms of the number of the prefix phrase plus the new bit needed to create the new phrase.

We will use parentheses and commas to separate these at first, in order to aid our visualization of the process.

The eight phrases can be described by:

(000,1),(000,0),(001,0),(001,1),(010,1),(011,1),(101,0),(110,1)

1, 0, 10, 11, 01, 101, 010, 1011

(000,1).(000,0).(001,0).(001,1).(010,1).(011,1).(101,0).(110,1)

- It can be read out as: (codeword at location 0.1).
(codeword at location 0.0). (codeword at location 1.0).
(codeword at location 1.1). (codeword at location 2.1).
(codeword at location 3.1) ...
- Thus the coded version of the above string is:

00010000001000110101011110101101

(000,1),(000,0),(001,0),(001,1),(010,1),(011,1),(101,0),(110,1)

Dictionary Location	Dictionary content	Fixed Length Codeword
001	1	0001
010	0	0000
011	10	0010
100	11	0011
101	01	0101
110	101	0111
111	010	1010
-	1011	1101

Length of table

- What should be the length of table?
- In practical applications, regardless of length of the table, it will eventually overflow
- This problem can be solved by pre-deciding a large enough size of dictionary.

- Data 1 0 11 01 00 010 0100 01001 0101
- Indexing 1 2 3 4 5 6 7 8 9
- Encode (0,1) (0,0) (1,1) (2,1) (2,0) (4,0) (6,0) (7,1) (8,1)

Code-book Index	Codebook entry	Code-book address
0	START	000
1	1	001
2	0	010
3	11	011
4	01	100
5	00	101
6	010	110
7	0100	111
8	0101	1000

Phrase	Previous entry coded	Previous entry code	Innovation Symbol	Encoded output
1	START	000	1	0001
0	START	000	0	0000
11	1	001	1	0011
01	0	010	1	0101
00	0	010	0	0100
010	01	100	0	1000
0100	010	110	0	1100
01001	0100	111	1	1111
0101	010	110	1	1101

- Received data 0001 0000 0011 0101 0100 1000 1100 1111 1101
- Index, I.S. 0,1 0,0 1,1 2,1 2,0 4,0 6,0 7,1 8,1
- Decoded o/p 1 0 11 01 00 010 0100 01001 0100

Determine the Lempel Ziv code for the following stream

01001111100101000001010101100111000

Recover the original sequence from the encoded stream

Determine the Lempel Ziv code for the following stream

1110100110000110100

Recover the original sequence from the encoded stream